

142. Ce qu'une ligne de C++ devient, en vrai

« `compte++` » n'existe pas pour la machine : elle ne connaît que lire, calculer, écrire.

1. Ce qu'il faut comprendre

Cliquer sur  **MODE MACHINE** juste après avoir compilé montre ce que la cartouche contient vraiment : des instructions du processeur, pas du C++. Ce mode se souvient d'où le compilateur a rangé chaque variable et chaque fonction — un « nom retrouvé » n'existe que pour une cartouche qui vient d'être compilée ici ; une cartouche venue d'ailleurs ne montre que des nombres.

`compte = 0` devient deux instructions : `xor a` (mettre le registre A à zéro) puis `ldh [main.compte], a` (l'écrire dans la case mémoire de `compte`). Il n'y a pas d'instruction « mettre une variable à zéro » sur ce processeur — seulement des registres et des adresses.

`compte++` n'est **pas** une seule instruction non plus : la machine doit LIRE la valeur (`ldh a, [main.compte]`), l'AUGMENTER dans un registre (`inc a`), puis la RÉÉCRIRE (`ldh [main.compte], a`). Trois instructions pour un `++`, parce que la puce ne calcule jamais directement en mémoire — tout passe par un registre.

Le `while (true)` du programme n'existe pas non plus dans la cartouche : c'est un simple retour en arrière, `jp`, qui saute à l'instruction d'avant plutôt que de continuer. La boucle est une manière humaine de LIRE le programme ; pour la machine, ce n'est qu'un saut.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **376 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	25 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireNombre — attend de pouvoir écrire une case
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles — 10 lettres sur 43

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

<code>nombre()</code> × 1	10 o sur place, dans <code>main()</code>	+ <code>EcrireNombre</code> (33 o)
<code>image()</code> × 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un compteur qui monte à chaque image. Il est tout petit
// exprès : on regarde ensuite, dans le MODE MACHINE, ce que chaque ligne
// devient pour le processeur.
//
//  compte = 0  → 2 instructions : mettre un registre à zéro, puis l'écrire en mémoire.
//  compte++   → 3 instructions : LIRE la case, AJOUTER 1, RÉÉCRIRE la case.
//  while (true) → un simple saut en arrière.

#include <nombre> // écrit un nombre en chiffres
```

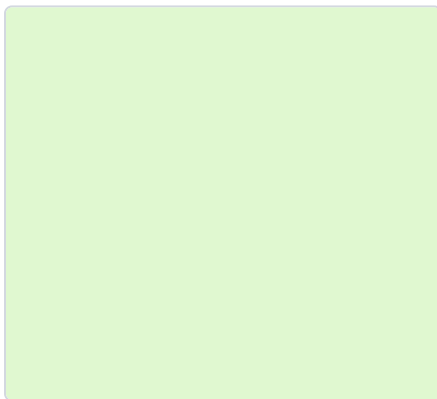
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t compte = 0;     // Une case « compte » qui commence à 0.

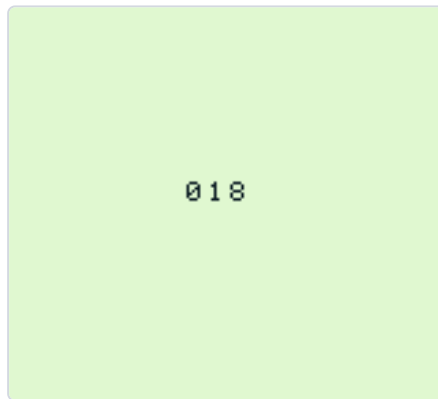
    while (true) {          // Répète pour toujours :
        compte++;           //   + 1,
        nombre(8, 8, compte); //   on l'affiche,
        image();            //   on attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

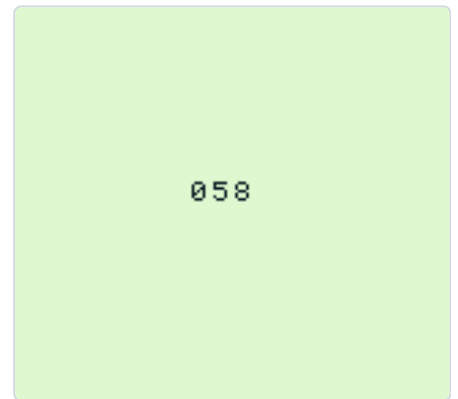
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un nombre qui monte à l'écran ; dans MODE MACHINE, chaque ligne de ce programme devient plusieurs instructions.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le compteur monte (18 → 58)

✓ et il est affiché

Ensuite : **143. Sinus, cosinus, une seule table** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++